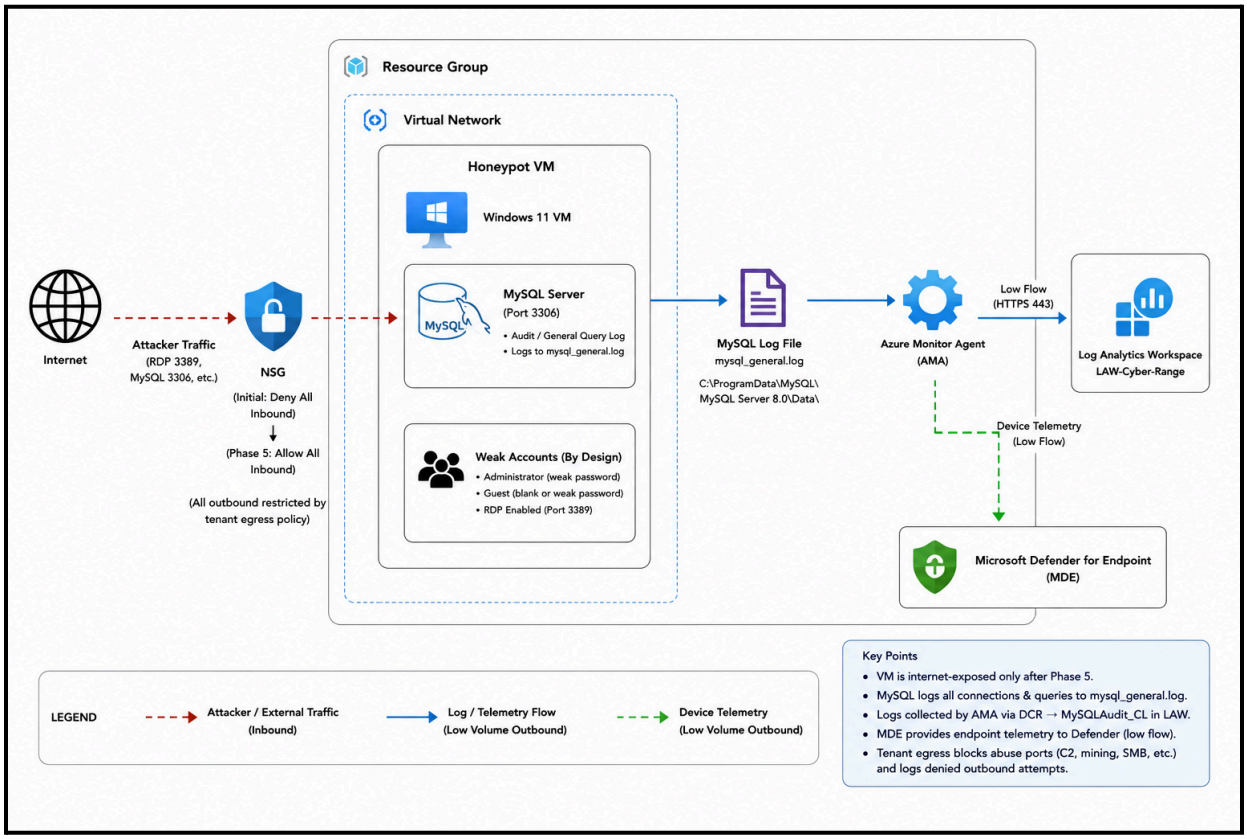
Cyber Range Capstone

Live-Exposed Honeypot Lab — End-to-End Checklist

Build hard → instrument → baseline → detect → weaken & expose. Windows VM + MySQL, telemetry to LAW-Cyber-Range.

This checklist walks the full defensive lifecycle for an internet-exposed asset. You build and harden the box, wire up logging, capture a clean baseline, write your detections while the environment is still quiet, and only then deliberately weaken and expose it so real attacker traffic supplies the breach. Work the phases in order — detections must exist before exposure so you catch your own incident.

Phase 0 — Honeypot Architecture



> **ENVIRONMENT NOTE**
> Workspace target is `LAW-Cyber-Range`. Tenant egress is heavily restricted centrally (allow-listed ports, known abuse ports explicitly denied), so a breached box is contained by design — attacker C2 / mining / pivoting attempts are blocked and logged rather than succeeding. Plan detections around denied / attempted outbound, not successful exfil.

Phase 1 — Build the VM Honeypot (locked down while we build it)

- ☐ Deploy a Windows 11 VM in your own resource group (strong username and password) — ensure you configure a public IP address
  - ☐ Name the VM something good, like “CORP-XXX-YYY” or something that looks legit, *not* “lab_test_1”
- ☐ Deny all inbound traffic from the internet
- ☐ Onboard the VM to Microsoft Defender for Endpoint (MDE), ensure it is showing up in the DeviceInfo table

Phase 2 — Install & populate MySQL

- ☐ On your VM, install Microsoft Visual C++ 2019 Redistributable Package (x64) (My SQL Requirement) (ref)
- ☐ On your VM Install MySQL (ref), Install with “Developer Default” or “Full” so it installs Workbench/the GUI
  - ☐ Install with all defaults
  - ☐ For the MySQL **root password**, on this phase, make it something strong, store it in a password manager for now (or somewhere you feel comfortable)
  - ☐ Everything else Defaults
- ☐ Start MySQL Workbench after the installation, create a new connection, and connect to MySQL
- ☐ Create a database and ingest dummy data → Download db_info_import.sql onto your VM and import it into your instance of MySQL (open SQL script, then execute it)
  - ☐ ⚠️this will seem like MySQL Workbench has frozen, it will just take some time. If the connection drops or it fails, just enter the MySQL password if needed and try again → change users from 5000 to 1000 or less if it fails too many times
- ☐ On the Schemas tab, refresh it to observe the new schema (database) “lnp_corp”
- ☐ **Enable MySQL audit / general query logging** so every connection (success and failure) and query is written to a log file. Do that by running this query:

```
SET GLOBAL general_log = 'ON';
SET GLOBAL log_output = 'FILE';
SHOW VARIABLES LIKE 'general_log%';
```

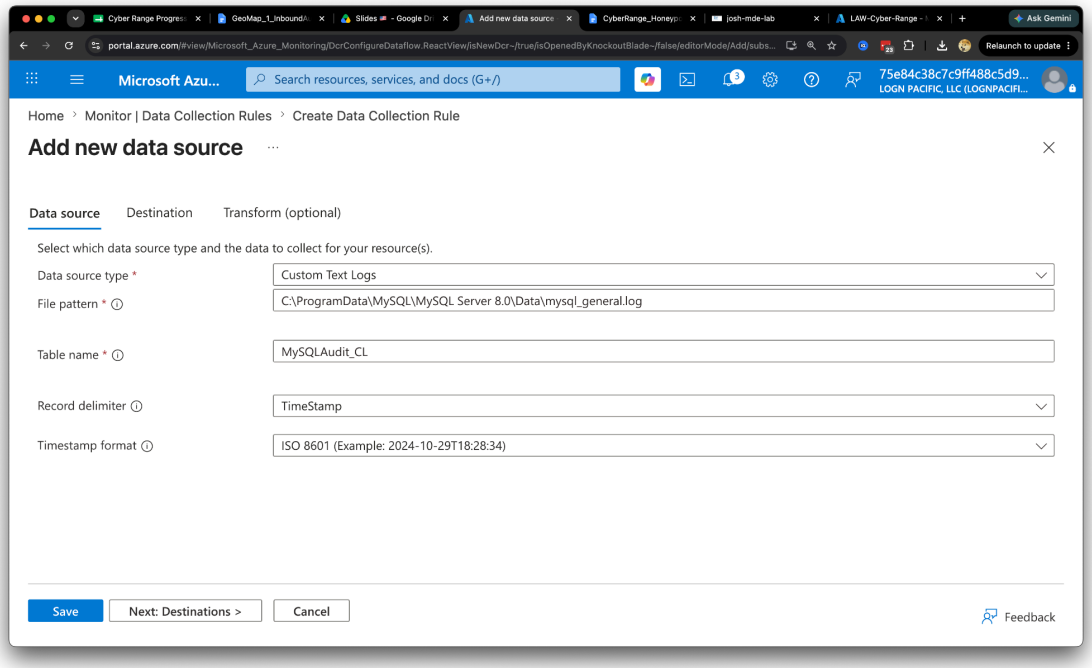
- ☐ Download my.ini and replace this file on your VM with it:**C:\ProgramData\MySQL\MySQL Server 8.0\my.ini**
  - ☐ This file tells MySQL to log everything to, it also allows login over the network: "**C:\ProgramData\MySQL\MySQL Server 8.0\Data\mysql_general.log**"
- ☐ Restart the MySQL80 service (services.msc)
- ☐ Run a few “SELECT” queries and then check the mysql_general.log to ensure the logs are coming through.
- ☐ Note the MySQL log file path (needed for the DCR in Phase 3)
- ☐ Bonus (not included in the video), create an entire backup of all databases in the server

Phase 3 — Wire logging to Log Analytics

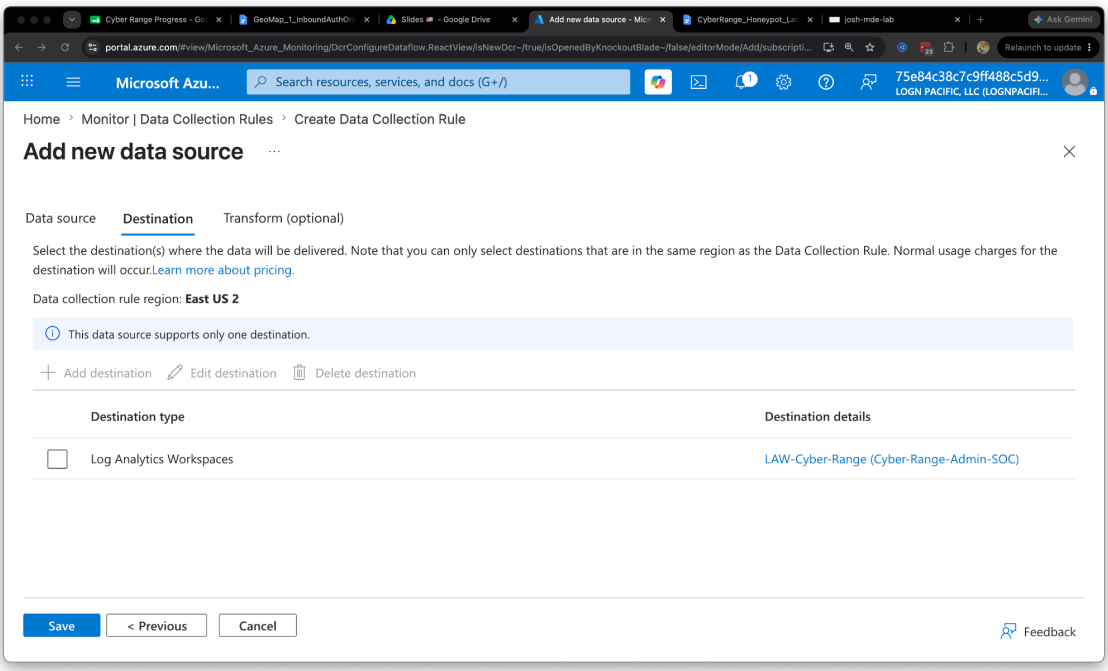
- ☐ Ensure your VM is ON and RUNNING, this is important

- [ ] Confirm device telemetry is flowing to `LAW-Cyber-Range` (check the DeviceInfo Table)
- [ ] Create a custom-text-log DCR pointing AMA at the MySQL log file → lands in a custom table: `MySQLAudit_CL`
- [ ] It's VERY important this is correct (Logs may take up to 30 minutes to appear)

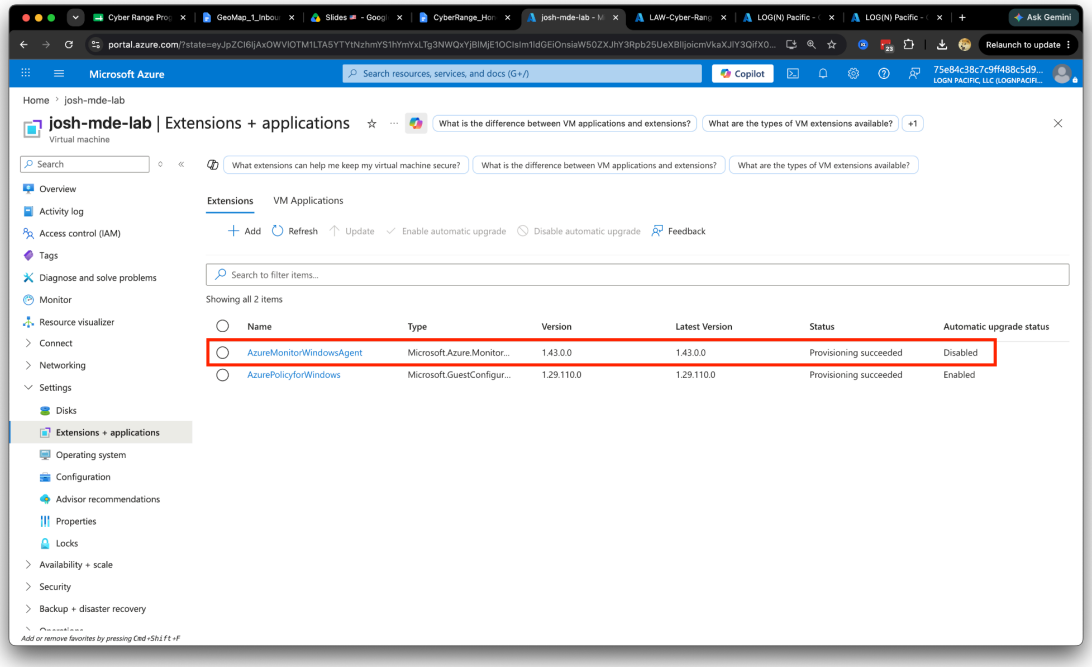
File pattern: C:\ProgramData\MySQL\MySQL Server 8.0\Data\mysql_general.log
Table name: MySQLAudit_CL
Record delimiter: TimeStamp
Timestamps format: ISO 8601



- [ ] For the Destination, choose LAW_Cyber_Range:



- [ ] Immediately after successfully creating the DCR, The Azure Monitor Agent will start to be installed on your VM. Browse to your VM → Settings → Extensions + applications, and look for “AzureMonitorWindowsAgent”:

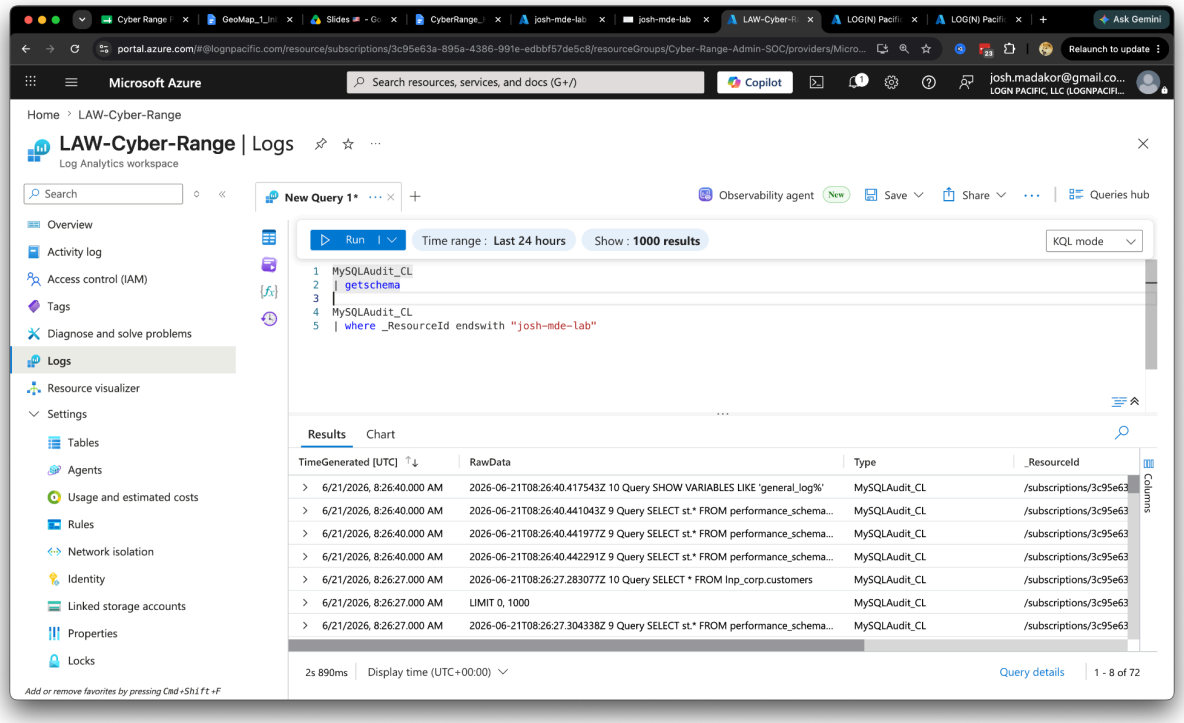


- [ ] Verify ingestion: query `MySQLAudit_CL` and confirm test connections / queries appear

```
MySQLAudit_CL
| project TimeGenerated, RawData, _ResourceId
| where _ResourceId endswith "your-VM-Name"
```

- [ ] Verify core device tables are populating. Once your logs are coming through, you can move on to Phase 4. ⚠️This table contains EVERYONE'S MySQL logs, you have to filter yours to match your ResourceId. For example:

```
MySQLAudit_CL
| where _ResourceId endswith "josh-mde-lab
```



Phase 4 — Write detections (while the box is still clean)

Author these as Sentinel analytics rules, before exposure, and confirm they are quiet against the clean baseline (no real successes yet).

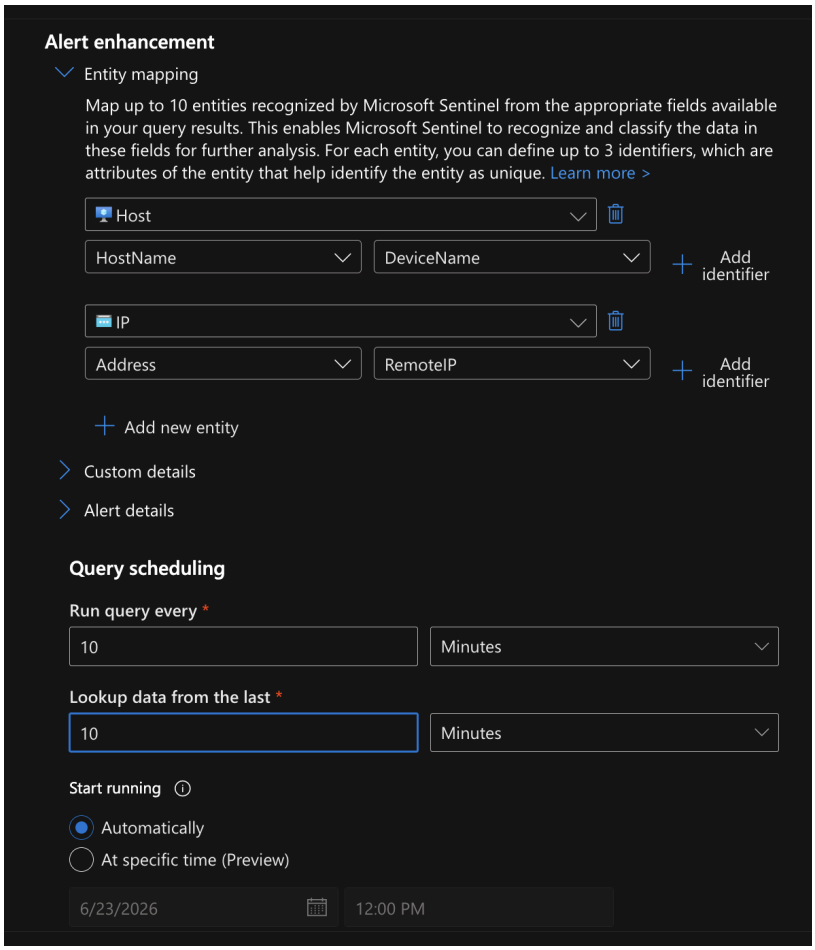
Rule — successful logon to the VM: `DeviceLogonEvents`, `ActionType == "LogonSuccess"`, public RemoteIP

- [x] ~~Create a Sentinel Analytics Rule for successful logons to the VM - Analytics Rule. name: _____~~

  For Example: Cyber-Defense-Final-corp-na01-fe123

```
// Virtual Machine Logons
let MyDevice = "corp-na01-fe123"; // MDE Truncates/cuts off the device name
DeviceLogonEvents
| where DeviceName == MyDevice
| where AccountName in~ ("administrator", "guest")
| where ActionType == "LogonSuccess"
| project TimeGenerated, RemoteIP, AccountName, DeviceName, ActionType,
LogonType
```

Fill in Entity Mapping and Analytics Rule settings



Rule — successful login to the MySQL database: `MySQLAudit_CL`, connect / auth-success events

When querying MySQLAudit_CL, you'll notice the logs come in the format of "RawData". These need to be parsed out into columns in order to work with them effectively. You can try to figure it out, or use this query to parse them out:

- [ ] **Create a Sentinel Analytics Rule for successful logons to the VM - Analytics Rule. name: _____**

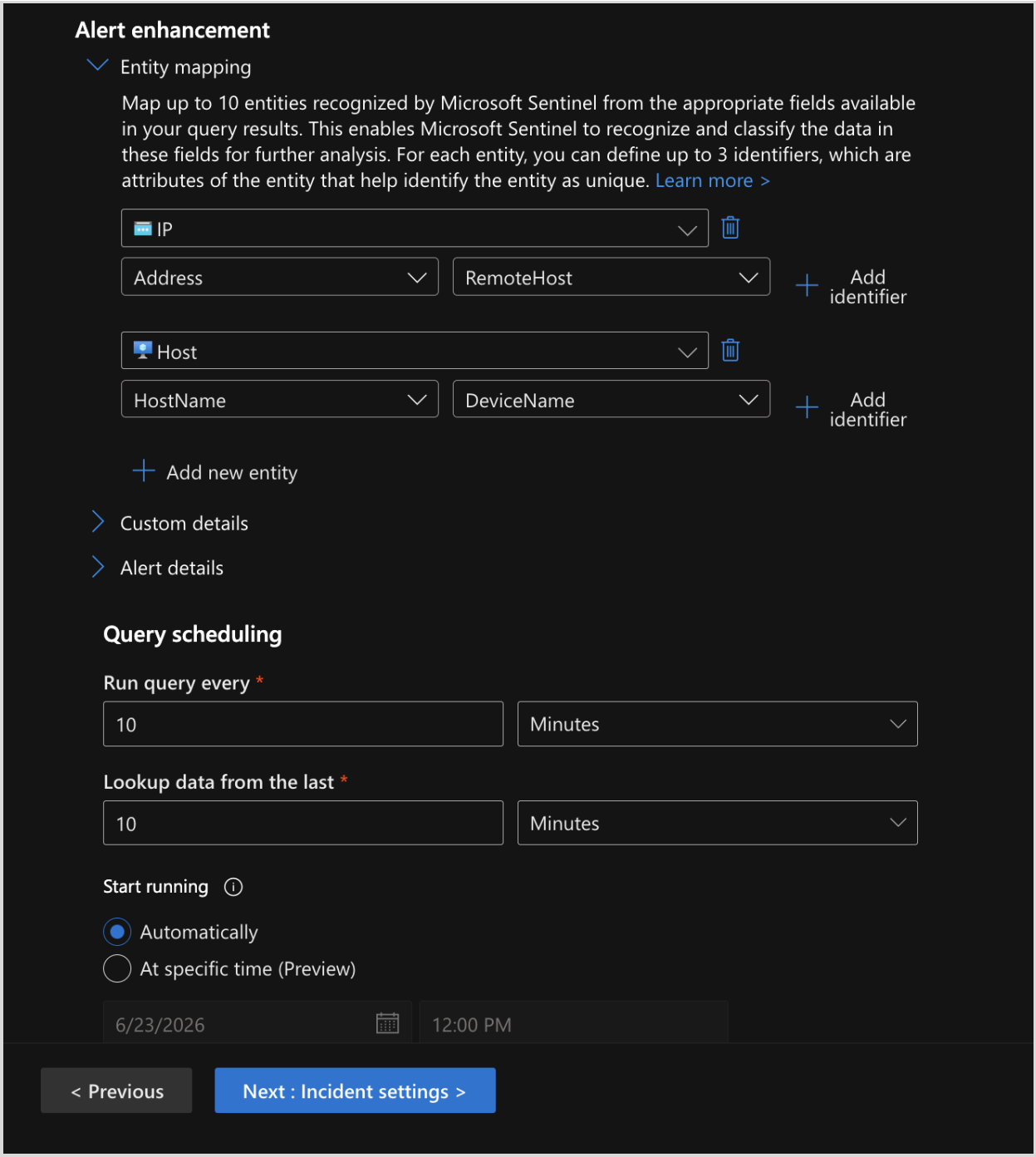
  For Example: Cyber-Defense-Final-MySQL

Rule query:

```
// SQL Server
let MyDevice = "corp-na01-fe123da";
let FailedConnections =
```

```
MySQLAudit_CL
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Access denied"
| extend ConnectionId = extract(@"^\S+\s+(\d+)\s+Connect", 1, RawData)
| distinct ConnectionId;
MySQLAudit_CL
| where TimeGenerated > MyTimeframe
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Connect"
| extend ConnectionId = extract(@"^\S+\s+(\d+)\s+Connect", 1, RawData)
| extend ActionType =
    case(
        RawData has "Access denied", "LogonFailure",
        ConnectionId in (FailedConnections), "Ignore",
        "LogonSuccess"
    )
| where ActionType != "Ignore"
| extend RawData = replace_string(RawData, "\t", " ")
| extend Username =
replace_string(tostring(split(tostring(split(RawData,"@")[0]), " ")[-1]), "'",
"")
| extend IpAddress = replace_string(tostring(split(split(RawData,"@")[1], "
")[0]), "'", "")
| where ActionType == "LogonSuccess"
| project TimeGenerated, DeviceName, Username, IpAddress, ActionType, RawData
| order by TimeGenerated desc
```

Fill in Entity Mapping and Analytics Rule settings



Phase 5 — Weaken & expose (deliberately, in order)

Only after detections are armed. Do these in sequence and record the exposure timestamp — it marks the start of the incident window.

- ☐ Via compmgmt.msc, enable (or create) the “Administrator” on your VM
  - ☐ Ensure it’s in the Administrators group, and set a weak password such as “password” or something from RockYou.txt top 10 weakest passwords
- ☐ Via compmgmt.msc, enable the “Guest” account and set the password to be be blank
  - ☐ Add the guest account to the “Users” group
- ☐ Allow the guest account to logon over the network: in secpol.msc → Local Policies → User Rights Assignment:
  - ☐ Deny log on through Remote Desktop Services — remove Guest from this list (it's there by default).
  - ☐ Allow log on through Remote Desktop Services — make sure Guest (or Remote Desktop Users) is listed.
  - ☐ Check Deny log on locally too, and remove Guest if present.
  - ☐ in the same console → Security Options: Set Accounts: Limit local account use of blank passwords to console logon only → Disabled (or just give Guest a real password, which is cleaner).
  - ☐ Under settings → Remote Desktop Users, add the Guest account
  - ☐ Run “gpupdate /force”

- ☐ Enable MySQL authentication reachable over the network; set MySQL root user to use “root” as the password as well:

```
CREATE USER 'root'@'%' IDENTIFIED BY 'root';
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' WITH GRANT OPTION;
FLUSH PRIVILEGES;
```

Note: this will create a SEPARATE root account with no password which can auth over the network

- ☐ **Capture an Investigation Package for your VM via Defender (Important), we will use this in our post breach analysis**
- ☐ Disable the Windows Firewall
- ☐ Weaken the NSG: allow ALL traffic inbound, increases discoverability
- ☐ **Record the exact exposure timestamp here: _**Aug 14, 2026 1:12:52 PM
  (we will use this to determine how long it took for the breach to happen)
- ☐ Confirm all detections are enabled and armed before walking away
- ☐ Your VM will automatically turn off at midnight EST every night, just keep turning it back on in the morning. This is unfortunate, but it’s for cost savings and soft breach containment

> **VECTOR DESIGN**
> Recommended path is RDP-breach-then-pivot: the box is brute-forced over RDP (the real attacker traffic you already see), the attacker then discovers and accesses the local MySQL and its dummy data. That yields a richer chain — initial access → discovery → collection — than a flat “someone hit 3306.” Exposing 3306 directly as well is the insurance policy if you want to guarantee the DB gets touched within a student’s window.

Phase 6 — Detect the Breach

Ensure your VM remains online and wait for your analytics rules to trigger.

Keep watching for logon activity against your VM and watching for incidents to be created in Defender/Sentinel based on your Analytics Rule. You can look through DeviceLogonEvents and MySQLAudit_CL, filtering for your device and looking at the latest logs.

For example, I might use the detection/analytics rule queries as helper queries to help me monitor activity

---

```
// SQL Server
let MyDevice = "corp-na01-fe123da";
let MyTimeframe = todatetime("2026-06-25T00:45:27.1820312Z");
let FailedConnections =
MySQLAudit_CL
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Access denied"
| extend ConnectionId = extract(@"^\S+\s+(\d+)\s+Connect", 1, RawData)
| distinct ConnectionId;
MySQLAudit_CL
| where TimeGenerated > MyTimeframe
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| where RawData has "Connect"
| extend ConnectionId = extract(@"^\S+\s+(\d+)\s+Connect", 1, RawData)
| extend ActionType =
    case(
        RawData has "Access denied", "LogonFailure",
        ConnectionId in (FailedConnections), "Ignore",
        "LogonSuccess"
    )
| where ActionType != "Ignore"
| extend RawData = replace_string(RawData, "\t", " ")
| extend Username =
replace_string(tostring(split(tostring(split(RawData,"@")[0]), " ")[-1]), "'",
"")
| extend IpAddress = replace_string(tostring(split(split(RawData,"@")[1], "
")[0]), "'", "")
| project TimeGenerated, DeviceName, Username, IpAddress, ActionType, RawData
| order by TimeGenerated desc
```

```
// Filtering Queries
let MyDevice = "corp-na01-fe123da"; // set your own device name
let ServerVulnerableDateTime = todatetime("2026-06-25T00:45:27.1820312Z");
MySQLAudit_CL
| where TimeGenerated > ServerVulnerableDateTime
| where RawData has "Query"
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| where DeviceName == MyDevice
| extend ActionType = "Query"
| extend Query = split(RawData, "Query")[1]
```

```
| project TimeGenerated, DeviceName, ActionType, Query, RawData
| order by TimeGenerated desc


// Virtual Machine Logons
let MyDevice = "corp-na01-fe123"; // MDE Truncates/cuts off the device name
let ServerVulnerableDateTime = todatetime("2026-06-25T00:45:27.1820312Z");
DeviceLogonEvents
| where TimeGenerated > ServerVulnerableDateTime
| where DeviceName == MyDevice
| where AccountName in~ ("administrator", "guest")
| project TimeGenerated, RemoteIP, AccountName, DeviceName, ActionType,
LogonType
```

When a breach finally happens, continue to monitor what the attacker is doing through the other tables.

A breach on the VM? Use: DeviceProcessEvents, DeviceFileEvents, DeviceNetworkEvents, etc, filter for your device as well as the “administrator” account.

You can also observe the NTANetAnalytics table to observe any traffic to/from your VM after the breach:

```
let MyDevice = "josh-mde-lab";
NTANetAnalytics
| where isnotempty(SrcVm)
| where SrcVm endswith MyDevice
| where DeniedOutFlows >= 1
| project TimeGenerated, DeviceName = MyDevice, FlowType, FlowStatus, SrcIp,
SrcPorts, DestIp, DestPort
```

A breach on MySQL? Use: MySQLAudit_CL to filter for commands, consider the following query:

```
// Filtering Queries
MySQLAudit_CL
| where RawData has "Query"
| extend RawData = replace_string(RawData, "\t", " ")
| extend DeviceName = tostring(split(_ResourceId, "/")[-1])
| extend ActionType = "Query"
| extend Query = split(RawData, "Query")[1]
| project TimeGenerated, DeviceName, ActionType, Query, RawData
| order by TimeGenerated desc
```

Phase 7 — Analyze the Breach

From here on out, we can't do a normal step-by-step instruction because what ends up happening to your environment is purely dynamic and will depend on when/how it gets breached.

After a day or so, assuming your VM/MySQL database is reachable, you should start getting indicators of compromise or signs of people poking around, trying to get in.

Open a new google doc to take notes / keep track of queries: Helper Queries (make a copy of this)

Keep Analyzing VM and MySQL authentication activity with the queries above

If someone is able to log into MySQL, keep an eye on the query log to see what kind of commands they are issuing.

If someone has been able to log into the virtual machine with **Guest** or **Administrator**, check the following tables:

- DeviceLogonEvents
- DeviceProcessEvents
- DeviceFileEvents
- DeviceRegistryEvents
- Or even DeviceNetworkEvents and NTANetAnalytics

Let the bad actor(s) sit for AT LEAST 24 hours or until you are satisfied with the data you have collected.

My VM and Database were online for at least 48 hours before the following happened and I decided to shut everything down.

Analyze the logs with ChatGPT:

MySQL Server Authentication Prompt:

You are a god-tier cybersecurity analyst with world-class threat hunting skills.
Please look at these logs, these are MySQL Server Authentication logs from my environment.
Please analyze them and paint a picture for what might be going on.

MySQL Server Query Prompt:

You are a god-tier cybersecurity analyst with world-class threat hunting skills.
Please look at these logs, these are MySQL Server Query logs from my environment.
Please analyze them and paint a picture for what might be going on.

Defender Logs Prompt:

You are a god-tier cybersecurity analyst with world-class threat hunting skills.
Please look at these logs, these are Microsoft Defender (MDE) logs from my environment.
Please analyze them and paint a picture for what might be going on.

When you have analyzed all of the logs and dumped your findings and note into your copy of Helper Queries and AI Analysis, you can move on to Phase 8

Phase 8 — Contain the Breach (Isolation)

Ensure your VM is on, Isolate it in the Defender Portal, and then capture another Investigation Package for your VM via Defender (Important), we will compare this to the first one we captured.

Capture the exact time of isolation here: Aug 19, 2026 11:02:39 AM

Ex: "2026-06-23T23:07:01.6859785Z"

Phase 9 — Eradication and Recovery

What this phase looks like depends on what happened in your environment and what was on the system that got compromised. Realistically for this, since both the VM and the MySQL Server got compromised, the best thing to do would be to simply destroy the VM and restore the database from backup.

Alternatively, if you didn't want to (or couldn't) destroy the VM, you could harden the system, harden the database, then restore the MySQL data from backup with the following steps:

- [ ] Harden the VM's NSG
- [ ] Turn on VM
- [ ] Remove VM from Isolation
- [ ] Run a full malware scan using Windows Defender
- [ ] Enable the Windows Firewall (wf.msc)
- [ ] Have no "administrator" account (Delete it)
- [ ] Leave the guest account Disabled (Disable it)
- [ ] Have a strong username/password for your local account
- [ ] Do not allow logon to the MySQL Server from the public internet
- [ ] Set a strong root password for the root account that logs in over the network (or delete it)
- [ ] "Restore" the data from backup (see Phase 2 — Install & populate MySQL)
    - [ ] Or if you took an actual backup, actually restore the data from backup

Phase 10 — Reporting

We build our end-to-end incident report.

Defender-Level Investigation

Export the following logs specifically for the time frame from your resource exposure time until isolation

- DeviceLogonEvents
- DeviceProcessEvents
- DeviceRegistryEvents
- DeviceNetworkEvents
- DeviceFileEvents
- MySQLAudit_CL (MySQLAudit_CL_Auth.csv + MySQLAuth_CL_Query.csv)
- NTANetAnalytics (or any other tables you feel are necessary)

Use your favorite LLM (I prefer Claude), upload the files, and use this prompt to formulate the incident report

Machine-Level Forensic Investigation

Use this prompt along with your captured investigation packages to see what has changed on the VM, before and after the breach.